ESP32 WiFi Dual Axis Motion Control

Wiring, web dashboard, program commands, reset program, and upload reference

Revision date: July 17, 2026

This document describes the current firmware and HTML interface for the ESP32 dual-axis controller. The web interface uses an orange theme based on #fd8208, has left-aligned compact panels, and includes separate Run Program and Reset Program editors.

1. System Overview

- Two DRV8825 stepper axes controlled with FastAccelStepper.
- X axis is Motor 1; Y axis is Motor 2.
- Home inputs and staged homing for both axes.
- Software travel limits and configurable unit conversion.
- Timed Gen.Out1 and Gen.Out2 dashboard outputs with millisecond duration entry.
- Program relay outputs R1, R2, and R3 for direct ON/OFF testing and program control.
- Sequential Program and Reset Program editors with line numbers and active-line tracking.
- Physical Run Program input on GPIO34 and physical Reset Program input on GPIO35.
- ESP32 AP mode plus optional router Wi-Fi station mode.

2. Hardware Connections

All signal grounds must be common: ESP32 GND, DRV8825 GND, relay module GND, and the control-side power supply ground.

Function	ESP32 GPIO	Connection
Motor 1 / X STEP	GPIO25	DRV8825 Motor 1 STEP
Motor 1 / X DIR	GPIO26	DRV8825 Motor 1 DIR
Motor 1 / X ENABLE	GPIO27	DRV8825 Motor 1 ENABLE
Motor 1 / X HOME	GPIO32	Home switch signal; other switch terminal to GND
Motor 1 / X M0	GPIO14	DRV8825 M0
Motor 1 / X M1	GPIO13	DRV8825 M1
Motor 1 / X M2	GPIO4	DRV8825 M2
Motor 2 / Y STEP	GPIO18	DRV8825 Motor 2 STEP
Motor 2 / Y DIR	GPIO19	DRV8825 Motor 2 DIR
Motor 2 / Y ENABLE	GPIO21	DRV8825 Motor 2 ENABLE
Motor 2 / Y HOME	GPIO33	Home switch signal; other switch terminal to GND
Motor 2 / Y M0	GPIO16	DRV8825 M0
Motor 2 / Y M1	GPIO17	DRV8825 M1
Motor 2 / Y M2	GPIO22	DRV8825 M2
Gen.Out1	GPIO23	Relay module input or transistor driver input
Gen.Out2	GPIO5	Relay module input or transistor driver input
R1 program relay	GPIO2	Relay module input or transistor driver input

Function	ESP32 GPIO	Connection
R2 program relay	GPIO12	Relay module input or transistor driver input
R3 program relay	GPIO15	Relay module input or transistor driver input
Run Program button	GPIO34	Momentary button to GND with external 10k pull-up to 3.3V
Reset Program button	GPIO35	Momentary button to GND with external 10k pull-up to 3.3V

GPIO34 and GPIO35 are input-only and do not have internal pull-up or pull-down resistors. Each button needs an external 10k pull-up to 3.3V, with the button connected between the GPIO and GND.

GPIO5, GPIO2, GPIO12, and GPIO15 are boot strapping pins on many ESP32 DevKit boards. Make sure relay modules or external circuitry do not force unsafe boot levels. If a relay input causes boot issues, move the related pin in main/Config.h or add suitable biasing/driver circuitry.

3. Output Behavior

The current output pin definitions are:

```
#define RELAY1_PIN                23
#define RELAY2_PIN                5
#define RELAY_ACTIVE_HIGH         true
#define PROGRAM_RELAY1_PIN        2
#define PROGRAM_RELAY2_PIN        12
#define PROGRAM_RELAY3_PIN        15
#define PROGRAM_START_BUTTON_PIN  34
#define RESET_PROGRAM_BUTTON_PIN  35
```

With RELAY_ACTIVE_HIGH true, outputs are ON when the GPIO is HIGH and OFF when LOW. For active-low relay modules, change RELAY_ACTIVE_HIGH to false in main/Config.h.

The firmware drives Gen.Out1 and Gen.Out2 to the inactive level as early as possible during setup, and the relay controller preloads the inactive output latch before enabling each GPIO as an output. This reduces firmware-controlled startup pulses.

If GPIO5 still pulses before firmware starts, that pulse is caused by ESP32 boot/strapping behavior outside firmware control. Solve it with hardware biasing, a transistor/driver stage, a relay module that does not react to boot leakage, or by moving Gen.Out2 to a safer free GPIO.

4. Wi-Fi Access

Field	Value
AP SSID	ESP32-MOTOR
AP Password	12345678
AP URL	http://192.168.4.1
Router / STA URL	Use the IP address assigned by the router, for example http://192.168.1.131

AP mode remains available even when router Wi-Fi credentials are saved. If the controller cannot join the router, connect directly to the ESP32-MOTOR access point.

5. Dashboard Tab

The Dashboard tab is the manual control and monitoring page. It contains compact, left-aligned control panels for X Axis - Motor 1, Y Axis - Motor 2, Timed General Outputs, Program Relay Manual Test, Driver Power Management, Motor Mechanical And Limit Configuration, System Information, and Console Log.

- Each motor panel shows position, target, remaining distance, progress, status, home switch state, and driver state.
- Move and jog controls can command each axis manually.
- Home starts the selected axis homing routine.
- Set Zero stores the current position as zero.
- Enable Driver and Disable Driver control driver power.
- Timed Gen.Out1 and Gen.Out2 duration values are entered and counted in milliseconds.
- Program Relay Manual Test provides direct ON/OFF buttons for R1, R2, and R3.
- All Stop stops motion, stops the active program, and turns off the Gen.Out outputs.

Timed Gen.Out default duration is 5000 ms. The maximum duration is 86400000 ms, which is 24 hours.

6. Motor Settings Tab

- Speed limit in steps per second.
- Acceleration in steps per second squared.
- Software minimum and maximum limits in raw steps.
- Step angle and microstep mode.

- Lead screw pitch, or belt pitch and pulley tooth count.
- Interface unit: pulse, mm, cm, or degree.
- Driver auto-disable timeout.

Settings are saved to ESP32 NVS and applied immediately. Program movement values for X and Y use the currently selected interface unit. VX and VY are always steps/s.

7. Program Tab

The Program tab has two independent editors: Sequential Program and Reset Program. They are displayed side by side in the web interface and use the same command language.

Program	Started By	Saved NVS Key	Purpose
Sequential Program	Run Program button or GPIO34	program_text	Normal line-by-line motion and output sequence
Reset Program	Reset Program button or GPIO35	reset_prog	Independent reset or recovery sequence

- Line numbers are shown next to each editor line.
- The status panel shows the active line while a program is running.
- A physical button start uses the latest saved program text.
- If no saved reset program exists, Reset Program will report that no reset program is saved.
- If one program is already running, a new Run Program or Reset Program request is ignored.
- The firmware waits for motor moves and homing cycles to finish before advancing.

Command Reference

Command	Description
X	Relative move for X axis / Motor 1. The value uses the selected interface unit.
Y	Relative move for Y axis / Motor 2. The value uses the selected interface unit.
VX	Changes the X axis runtime speed in steps/s.
VY	Changes the Y axis runtime speed in steps/s.
X HOME	Starts X axis homing and waits until it finishes.
Y HOME	Starts Y axis homing and waits until it finishes.
D	Waits for the specified number of milliseconds.
WAIT	Readable alias for D.
Gen1 ON / OFF	Turns Gen.Out1 on or off.
Gen2 ON / OFF	Turns Gen.Out2 on or off.
R1 ON / OFF	Turns R1 program relay output on GPIO2 on or off.
R2 ON / OFF	Turns R2 program relay output on GPIO12 on or off.
R3 ON / OFF	Turns R3 program relay output on GPIO15 on or off.
# or //	Starts a comment. Empty lines are ignored.

VX and VY update runtime motor speed. During X HOME and Y HOME, the homing routine uses staged behavior: the search stage uses the configured/runtime motor speed limit, and the final approach remains intentionally slower for repeatable home detection.

Example Sequential Program

```
VX 5000
VY 5000
X 50
Y 10
D 50
Gen1 ON
R1 ON
D 100
X -50
Gen2 ON
D 100
R1 OFF
Gen1 OFF
Gen2 OFF
```

This example sets X and Y runtime speed to 5000 steps/s, moves X and Y, waits in milliseconds, switches Gen.Out and relay outputs, and then turns the outputs back off.

Example Reset Program

```
VX 8000
VY 8000
Gen1 OFF
Gen2 OFF
R1 OFF
R2 OFF
R3 OFF
X HOME
Y HOME
D 100
X 10
Y 10
```

This reset example turns outputs off, homes both axes, waits 100 ms, and moves both axes away from home by 10 selected interface units.

8. Build And Upload

- Install the ESP32 board package in Arduino IDE or Arduino CLI.
- Select an ESP32 Dev Module compatible board profile.
- Install FastAccelStepper, ESPAsyncWebServer, AsyncTCP, and ArduinoJson.
- Open main/main.ino.
- Confirm pin assignments in main/Config.h.
- Compile and upload to the ESP32.

```
arduino-cli compile --fqbn esp32:esp32:esp32 main
arduino-cli upload -p COM6 --fqbn esp32:esp32:esp32 main
```

9. Local Versioning And GitHub

Before larger changes, full local project snapshots are kept under legacy/. The legacy folder is intentionally ignored by Git and must not be pushed. Current source files and this PDF are pushed to GitHub.

10. Safety Notes

- Set the DRV8825 current limit before running motors.
- Run first tests with low speed and low acceleration.
- Use relay modules or proper transistor drivers; do not connect relay coils directly to ESP32 GPIO pins.
- Use fusing, isolation, and proper enclosure practices for AC or high-current loads.
- Firmware controls are not a complete machine safety system. Add a hardware emergency stop where required.
- Test home switch wiring before starting mechanical motion.